Uploaded a file

I would like to solve for the best value of a and b in
f(x) = a ln(N) * M * K^b
a > 0 and 0.5 < b < 1

for data that looks like the following:

N=100000
Build M=8 ef=128 in 11.71s with 0.69 short edges
 Query PQ=false top 101/1 recall 0.9061 in 0.91s after 10017287 nodes visited
 Query PQ=false top 101/2 recall 0.9554 in 1.45s after 17204099 nodes visited
 Query PQ=false top 101/4 recall 0.9733 in 2.71s after 30042785 nodes visited
Build M=16 ef=128 in 16.88s with 0.42 short edges
 Query PQ=false top 101/1 recall 0.9477 in 1.25s after 15407385 nodes visited
 Query PQ=false top 101/2 recall 0.9715 in 2.26s after 26048892 nodes visited
 Query PQ=false top 101/4 recall 0.9821 in 4.07s after 44803171 nodes visited

There will be multiple blocks starting with an "N=" line.

In each block, there will be multiple Build lines, containing an M.

For each Build line, there will be multiple Query lines. Extract "top X/Y" from each query line; K=X*Y. Finally, f(x) is Z/10000 in "Z nodes visited" in the query line.

To find the best values for a and b in the function $f(x) = a \ln(N) \cdot M \cdot K^b$, where $f(x)$ is given by $Z/10000$ ("Z nodes visited" in the query line), we can proceed as follows:

1. Parse the input data to extract values for N, M, K, and $f(x)$.
2. For each query line, compute K as the product of the numbers extracted from "top X/Y", and compute $f(x)$ as the number Z divided by 10000.
3. Use the extracted data to fit the function $f(x) = a\ln(N) \cdot M \cdot K^b$ to find the best values for a and b. This can be done using nonlinear least squares fitting.

Now, let's start by implementing step 1 to parse the data and extract the relevant values.

Analyzed

python Always show details Copy code

```python
from typing import List, Tuple
import re

# Define a function to parse the data
def parse_data(file_path: str) -> List[Tuple[int, int, float, float]]:
    """
    Parse the data from the input file and extract the values for N, M, K,

    Args:
    file_path (str): The path to the input file.

    Returns:
    List[Tuple[int, int, float, float]]: A list of tuples containing the va
    """
    data = []
    with open(file_path, 'r') as file:
        lines = file.readlines()

    N = 0
    M = 0
    for line in lines:
        if line.startswith('N='):
            N = int(line.split('=')[1])
        elif line.startswith('Build M='):
            M = int(re.search(r'Build M=(\d+)', line).group(1))
        elif line.startswith('  Query'):
            top_values = re.search(r'top (\d+)/(\d+)', line)
            X = int(top_values.group(1))
            Y = int(top_values.group(2))
```

```
                K = X * Y
                Z = int(re.search(r'(\d+) nodes visited', line).group(1))
                f_x = Z / 10000
                data.append((N, M, K, f_x))
    return data

# Parse the data from the input file
data = parse_data('/mnt/data/out2.txt')
data
```

Result

```
[(100000, 8, 2, 117.7497),
 (100000, 8, 4, 171.3892),
 (100000, 8, 8, 229.1776),
 (100000, 8, 16, 310.8159),
 (100000, 8, 32, 450.2437),
 (100000, 8, 64, 711.3031),
 (100000, 8, 100, 987.2987),
 (100000, 8, 200, 1696.2117),
 (100000, 8, 400, 2974.8185),
 (100000, 12, 2, 154.8259),
 (100000, 12, 4, 194.1064),
 (100000, 12, 8, 248.9131),
 (100000, 12, 16, 336.4),
 (100000, 12, 32, 560.5212),
 (100000, 12, 64, 898.29),
 (100000, 12, 100, 1253.144),
 (100000, 12, 200, 2160.2542),
 (100000, 12, 400, 3764.9767),
 (100000, 16, 2, 197.0804),
 (100000, 16, 4, 252.2298),
 (100000, 16, 8, 323.2802),
 (100000, 16, 16, 439.824),
 (100000, 16, 32, 653.7488),
 (100000, 16, 64, 1053.8146),
 (100000, 16, 100, 1471.9113),
 (100000, 16, 200, 2535.9635),
 (100000, 16, 400, 4406.8987),
 (100000, 20, 2, 275.4544),
 (100000, 20, 4, 335.9144),
```

```
    (100000, 20, 8, 420.037),
    (100000, 20, 16, 557.4703),
    (100000, 20, 32, 802.9427),
    (100000, 20, 64, 1257.0709),
    (100000, 20, 100, 1727.181),
    (100000, 20, 200, 2921.2304),
    (100000, 20, 400, 4995.5601),
    (289761, 16, 2, 192.3282),
    (289761, 16, 4, 281.2171),
    (289761, 16, 8, 418.329),
    (289761, 16, 16, 632.8463),
    (289761, 16, 32, 987.3547),
    (289761, 16, 64, 1603.9504),
    (289761, 16, 100, 2246.2841),
    (289761, 16, 200, 3986.3759),
    (289761, 16, 400, 7487.803),
    (289761, 24, 2, 274.1708),
    (289761, 24, 4, 394.9698),
    (289761, 24, 8, 576.1592),
    (289761, 24, 16, 860.6284),
    (289761, 24, 32, 1336.3753),
    (289761, 24, 64, 2179.3581),
    (289761, 24, 100, 3085.3052),
    (289761, 24, 200, 5575.5448),
    (289761, 24, 400, 10605.4978),
    (289761, 32, 2, 371.3996),
    (289761, 32, 4, 509.9509),
    (289761, 32, 8, 719.6302),
    (289761, 32, 16, 1053.4047),
    (289761, 32, 32, 1626.455),
    (289761, 32, 64, 2673.2056),
    (289761, 32, 100, 3819.5912),
    (289761, 32, 200, 7004.2204),
    (289761, 32, 400, 13432.8053),
    (1183514, 16, 2, 231.474),
    (1183514, 16, 4, 346.8718),
    (1183514, 16, 8, 515.8684),
    (1183514, 16, 16, 773.773),
    (1183514, 16, 32, 1192.8685),
    (1183514, 16, 64, 1923.8344),
```

```
    (1183514, 16, 100, 2673.1855),
    (1183514, 16, 200, 4596.8872),
    (1183514, 16, 400, 8155.6008),
    (1183514, 24, 2, 336.4613),
    (1183514, 24, 4, 482.0456),
    (1183514, 24, 8, 695.3627),
    (1183514, 24, 16, 1034.2001),
    (1183514, 24, 32, 1592.5289),
    (1183514, 24, 64, 2593.8196),
    (1183514, 24, 100, 3642.6325),
    (1183514, 24, 200, 6368.8057),
    (1183514, 24, 400, 11425.4723),
    (1183514, 32, 2, 444.9436),
    (1183514, 32, 4, 621.9555),
    (1183514, 32, 8, 879.6657),
    (1183514, 32, 16, 1286.4182),
    (1183514, 32, 32, 1988.5114),
    (1183514, 32, 64, 3254.8304),
    (1183514, 32, 100, 4585.1152),
    (1183514, 32, 200, 8057.1531),
    (1183514, 32, 400, 14484.0307),
    (1000000, 16, 2, 274.5925),
    (1000000, 16, 4, 342.1041),
    (1000000, 16, 8, 447.5537),
    (1000000, 16, 16, 628.9684),
    (1000000, 16, 32, 960.7225),
    (1000000, 16, 64, 1579.2115),
    (1000000, 16, 100, 2223.8133),
    (1000000, 16, 200, 3831.6719),
    (1000000, 16, 400, 6610.737),
    (1000000, 24, 2, 368.24),
    (1000000, 24, 4, 459.3679),
    (1000000, 24, 8, 604.4545),
    (1000000, 24, 16, 858.8728),
    (1000000, 24, 32, 1329.6902),
    (1000000, 24, 64, 2197.6978),
    (1000000, 24, 100, 3086.2674),
    (1000000, 24, 200, 5269.4607),
    (1000000, 24, 400, 8976.8057),
    (1000000, 32, 2, 466.9022),
```

```
(1000000, 32, 4, 579.2008),
(1000000, 32, 8, 757.6607),
(1000000, 32, 16, 1078.8349),
(1000000, 32, 32, 1672.7429),
(1000000, 32, 64, 2756.0965),
(1000000, 32, 100, 3853.176),
(1000000, 32, 200, 6520.8995),
(1000000, 32, 400, 11009.8286),
(60000, 8, 2, 139.8401),
(60000, 8, 4, 165.4042),
(60000, 8, 8, 202.2152),
(60000, 8, 16, 265.859),
(60000, 8, 32, 377.4229),
(60000, 8, 64, 565.4869),
(60000, 8, 100, 747.8028),
(60000, 8, 200, 1178.0993),
(60000, 8, 400, 1892.8574)]
```

We have successfully parsed the data and extracted values for N, M, K, and $f(x)$.

The next step is to use nonlinear least squares fitting to find the best values for a and b that minimize the error in predicting $f(x)$ using the equation $f(x) = a \ln(N) \cdot M \cdot K^b$.

Let's proceed with this step. ‹/›

Analyzed ^

python Always show details | Copy code

```python
from scipy.optimize import curve_fit
import numpy as np

# Define the function form
def func(x, a, b):
    """
    Define the function f(x) = a ln(N) * M * K^b

    Args:
    x (numpy.ndarray): An array containing the values for N, M, and K.
```

```
    a (float): The parameter a in the function.
    b (float): The parameter b in the function.

    Returns:
    numpy.ndarray: The values of f(x) computed using the input values and p
    """
    N, M, K = x
    return a * np.log(N) * M * K**b

# Prepare the data for fitting
N = np.array([item[0] for item in data])
M = np.array([item[1] for item in data])
K = np.array([item[2] for item in data])
f_x = np.array([item[3] for item in data])

# Fit the function to the data using nonlinear least squares fitting
params, covariance = curve_fit(func, (N, M, K), f_x, bounds=([0, 0.5], [np

# Get the best values for a and b
a_best, b_best = params
a_best, b_best
```

Result

```
(0.2600907517986047, 0.7933338409162497)
```

The best-fit values for the parameters are approximately:

- $a \approx 0.2601$
- $b \approx 0.7933$

These values minimize the error in predicting $f(x)$ with the given equation $f(x) = a \ln(N) \cdot M \cdot K^b$ based on the provided data set.

how accurate is our approximation, as a percentage of the observed values?

Analyzed

python Always show details Copy code

```python
# Calculate the predicted values using the best-fit parameters
f_x_predicted = func((N, M, K), a_best, b_best)

# Calculate the relative error as a percentage
relative_error_percentage = np.abs((f_x - f_x_predicted) / f_x) * 100

# Calculate the mean and standard deviation of the relative error percentag
mean_relative_error_percentage = np.mean(relative_error_percentage)
std_relative_error_percentage = np.std(relative_error_percentage)

mean_relative_error_percentage, std_relative_error_percentage
```

Result

```
(23.709966315052228, 18.689413771798208)
```

The approximation has a mean relative error of approximately 23.71%, with a standard deviation of about 18.69%. This indicates that on average, the predicted values deviate from the observed values by about 23.71%, with a variation in the error given by the standard deviation.